

| <b>DSA0502 – Query Processing for Data Science</b> |                       |
|----------------------------------------------------|-----------------------|
| Course Faculty                                     | Dr. B .SHYAMALA GOWRI |
| Reg. No.                                           | 192324164             |
| Student Name                                       | Sweetie Roselin A     |
| Date                                               | 30/07/2026            |

## **CO2 AT1 – DEIGN AND CONNECTIVITY**

### **Scenario 1: University Student Management System**

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

**Table 1: Department**

| Column Name    | Data Type | Key       |
|----------------|-----------|-----------|
| DepartmentID   | INTEGER   | <b>PK</b> |
| DepartmentName | TEXT      |           |
| HOD            | TEXT      |           |

**Table 2: Student**

| Column Name | Data Type | Key       |
|-------------|-----------|-----------|
| StudentID   | INTEGER   | <b>PK</b> |
| Name        | TEXT      |           |
| Gender      | TEXT      |           |
| DOB         | DATE      |           |
| Phone       | TEXT      |           |

|              |         |                                      |
|--------------|---------|--------------------------------------|
| Email        | TEXT    |                                      |
| DepartmentID | INTEGER | <b>FK → Department(DepartmentID)</b> |

**Table 3: Faculty**

| Column Name   | Data Type | Key                                  |
|---------------|-----------|--------------------------------------|
| FacultyID     | INTEGER   | <b>PK</b>                            |
| FacultyName   | TEXT      |                                      |
| Qualification | TEXT      |                                      |
| Phone         | TEXT      |                                      |
| DepartmentID  | INTEGER   | <b>FK → Department(DepartmentID)</b> |

**Table 4: Course**

| Column Name | Data Type | Key                            |
|-------------|-----------|--------------------------------|
| CourseID    | INTEGER   | <b>PK</b>                      |
| CourseName  | TEXT      |                                |
| Credits     | INTEGER   |                                |
| FacultyID   | INTEGER   | <b>FK → Faculty(FacultyID)</b> |

**Table 5: Enrollment**

| Column Name  | Data Type | Key                            |
|--------------|-----------|--------------------------------|
| EnrollmentID | INTEGER   | <b>PK</b>                      |
| StudentID    | INTEGER   | <b>FK → Student(StudentID)</b> |
| CourseID     | INTEGER   | <b>FK → Course(CourseID)</b>   |
| Semester     | TEXT      |                                |

**Table 6: Attendance**

| Column Name          | Data Type | Key                            |
|----------------------|-----------|--------------------------------|
| AttendanceID         | INTEGER   | <b>PK</b>                      |
| StudentID            | INTEGER   | <b>FK → Student(StudentID)</b> |
| CourseID             | INTEGER   | <b>FK → Course(CourseID)</b>   |
| AttendancePercentage | REAL      |                                |

**Table 7: Examination**

| Column Name | Data Type | Key                            |
|-------------|-----------|--------------------------------|
| ExamID      | INTEGER   | <b>PK</b>                      |
| StudentID   | INTEGER   | <b>FK → Student(StudentID)</b> |
| CourseID    | INTEGER   | <b>FK → Course(CourseID)</b>   |
| Marks       | INTEGER   |                                |
| Grade       | TEXT      |                                |

## 2. Relationships

- Department → Student (**1 : Many**)
- Department → Faculty (**1 : Many**)
- Faculty → Course (**1 : Many**)
- Student ↔ Course (**Many : Many**) through Enrollment
- Student → Attendance (**1 : Many**)
- Student → Examination (**1 : Many**)

## 3. Primary Keys & Foreign Keys

| Table       | Primary Key  | Foreign Key         |
|-------------|--------------|---------------------|
| Department  | DepartmentID | —                   |
| Student     | StudentID    | DepartmentID        |
| Faculty     | FacultyID    | DepartmentID        |
| Course      | CourseID     | FacultyID           |
| Enrollment  | EnrollmentID | StudentID, CourseID |
| Attendance  | AttendanceID | StudentID, CourseID |
| Examination | ExamID       | StudentID, CourseID |

## SQLite Database using Python

```
import sqlite3
```

```
conn = sqlite3.connect("university.db")
```

```
cursor = conn.cursor()
```

```
cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS Department(
```

```
DepartmentID INTEGER PRIMARY KEY,
```

```

DepartmentName TEXT,
HOD TEXT
)
""")
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
StudentID INTEGER PRIMARY KEY,
Name TEXT,
Gender TEXT,
DOB TEXT,
Phone TEXT,
Email TEXT,
DepartmentID INTEGER,
FOREIGN KEY(DepartmentID) REFERENCES Department(DepartmentID)
)
""")
conn.commit()

```

## **CRUD Operations**

### **Create (Insert)**

```

cursor.execute("INSERT INTO Department VALUES (1,'CSE','Dr. Kumar')")
cursor.execute("""
INSERT INTO Student
VALUES
(101,'John','Male','2005-03-12','9876543210',
'john@gmail.com',1)
""")
conn.commit()

```

### **Read (Select)**

```

cursor.execute("SELECT * FROM Student")

```

```
for row in cursor.fetchall():  
    print(row)
```

### **Update**

```
cursor.execute("""  
UPDATE Student  
SET Phone='9999999999'  
WHERE StudentID=101  
""")  
conn.commit()
```

### **Delete**

```
cursor.execute("""  
DELETE FROM Student  
WHERE StudentID=101  
""")  
conn.commit()  
conn.close()
```

### **Expected Reports**

- Department-wise Student Strength

```
SELECT DepartmentName, COUNT(StudentID)  
FROM Department  
JOIN Student  
ON Department.DepartmentID=Student.DepartmentID  
GROUP BY DepartmentName;
```

- Faculty Workload

```
SELECT FacultyID, COUNT(CourseID)  
FROM Course  
GROUP BY FacultyID;
```

- Student Performance

```
SELECT StudentID, AVG(Marks)
```

FROM Examination

GROUP BY StudentID;

```
... === REPORT 1: Department-wise Student Strength ===
DepartmentName StudentCount
           CSE             2
           ECE             1

=== REPORT 2: Faculty Workload ===
FacultyID CourseCount
      201             2
      202             1

=== REPORT 3: Student Performance ===
StudentID AverageMarks
      101          87.5
      102          91.5
      103          78.0
```

### Scenario 6: Banking and Customer Account Management System

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management

**Table 1: Customer**

| Column Name  | Data Type | Key       |
|--------------|-----------|-----------|
| CustomerID   | INTEGER   | <b>PK</b> |
| CustomerName | TEXT      |           |
| Address      | TEXT      |           |
| Phone        | TEXT      |           |
| Email        | TEXT      |           |

**Table 2: Branch**

| Column Name | Data Type | Key       |
|-------------|-----------|-----------|
| BranchID    | INTEGER   | <b>PK</b> |
| BranchName  | TEXT      |           |
| Location    | TEXT      |           |

**Table 3: Account**

| Column Name   | Data Type | Key                              |
|---------------|-----------|----------------------------------|
| AccountNumber | INTEGER   | <b>PK</b>                        |
| AccountType   | TEXT      |                                  |
| Balance       | REAL      |                                  |
| CustomerID    | INTEGER   | <b>FK → Customer(CustomerID)</b> |
| BranchID      | INTEGER   | <b>FK → Branch(BranchID)</b>     |

**Table 4: Transaction**

| Column Name     | Data Type | Key                                |
|-----------------|-----------|------------------------------------|
| TransactionID   | INTEGER   | <b>PK</b>                          |
| AccountNumber   | INTEGER   | <b>FK → Account(AccountNumber)</b> |
| TransactionType | TEXT      |                                    |
| Amount          | REAL      |                                    |
| TransactionDate | DATE      |                                    |

**Table 5: Loan**

| Column Name     | Data Type | Key                              |
|-----------------|-----------|----------------------------------|
| LoanID          | INTEGER   | <b>PK</b>                        |
| CustomerID      | INTEGER   | <b>FK → Customer(CustomerID)</b> |
| LoanAmount      | REAL      |                                  |
| LoanType        | TEXT      |                                  |
| RepaymentStatus | TEXT      |                                  |

**2. Relationships**

- Customer → Account (**1 : Many**)
- Branch → Account (**1 : Many**)
- Account → Transaction (**1 : Many**)
- Customer → Loan (**1 : Many**)

### 3. Primary Keys & Foreign Keys

| Table       | Primary Key   | Foreign Key          |
|-------------|---------------|----------------------|
| Customer    | CustomerID    | —                    |
| Branch      | BranchID      | —                    |
| Account     | AccountNumber | CustomerID, BranchID |
| Transaction | TransactionID | AccountNumber        |
| Loan        | LoanID        | CustomerID           |

#### Program

```
import mysql.connector

# Connect to MySQL Server

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="your_password"
)

cursor = conn.cursor()

# Create Database
cursor.execute("CREATE DATABASE IF NOT EXISTS bankdb")
cursor.execute("USE bankdb")

# Create Customer Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    Address VARCHAR(200),
    Phone VARCHAR(15),
```



```
Email VARCHAR(100)
)
""")
```

```
# Create Branch Table
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Branch(
    BranchID INT PRIMARY KEY,
    BranchName VARCHAR(100),
    Location VARCHAR(100)
)
""")
```

```
# Create Account Table
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Account(
    AccountNumber INT PRIMARY KEY,
    AccountType VARCHAR(50),
    Balance DECIMAL(10,2),
    CustomerID INT,
    BranchID INT,
    FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY(BranchID) REFERENCES Branch(BranchID)
)
""")
```

```
conn.commit()
```

```
# -----
```

```
# CRUD Operations
```

```
# -----
```

```
# CREATE (INSERT)

cursor.execute("""

INSERT INTO Customer

VALUES

(1,'Alice','Chennai','9876543210','alice@gmail.com')

""")
```

```
cursor.execute("""

INSERT INTO Branch

VALUES

(101,'Anna Nagar Branch','Chennai')

""")
```

```
cursor.execute("""

INSERT INTO Account

VALUES

(5001,'Savings',25000.00,1,101)

""")
```

```
conn.commit()
```

```
# -----
```

```
# READ (SELECT)
```

```
# -----
```

```
cursor.execute("SELECT * FROM Account")
```

```
rows = cursor.fetchall()
```

```
for row in rows:
```

```
    print(row)
```

```
# -----
```

```
# UPDATE

# -----

cursor.execute("""

UPDATE Account

SET Balance = 30000

WHERE AccountNumber = 5001

""")
```

```
conn.commit()
```

```
# -----

# DELETE

# -----

cursor.execute("""

DELETE FROM Account

WHERE AccountNumber = 5001

""")
```

```
conn.commit()
```

```
cursor.close()
```

```
conn.close()
```

## **Expected Reports**

### **1. Customer Balances**

```
SELECT Customer.CustomerName, Account.Balance

FROM Customer

JOIN Account

ON Customer.CustomerID = Account.CustomerID;
```

### **2. Transaction History**

*(Assuming a Transaction table exists.)*

```
SELECT *

FROM Transaction
```

WHERE AccountNumber = 5001;

### 3. Loan Repayment Status

SELECT Customer.CustomerName,

Loan.LoanAmount,

Loan.RepaymentStatus

FROM Customer

JOIN Loan

ON Customer.CustomerID = Loan.CustomerID;

```
▼ ... === REPORT 1: Customer Balances ===
CustomerName Balance
Alice 30000.0
Bob 50000.0

=== REPORT 2: Transaction History (AccountNumber = 5001) ===
TransactionID AccountNumber TransactionType Amount TransactionDate
1001 5001 Deposit 5000.0 2026-07-01
1002 5001 Withdrawal 2000.0 2026-07-05

=== REPORT 3: Loan Repayment Status ===
CustomerName LoanAmount RepaymentStatus
Alice 500000.0 Active / On-Time
Bob 200000.0 Closed
```